



Esercitazione 9

Filtraggio di immagini



Introduzione

In questa esercitazione studiamo il filtraggio di immagini nel **dominio delle frequenze** usando la Trasformata Discreta di Fourier 2-D. L'idea chiave è che un'immagine può essere rappresentata come somma di componenti a diverse frequenze spaziali: le basse frequenze codificano la struttura globale e le variazioni lente (aree uniformi, luminosità generale), le alte frequenze contengono dettagli fini e bordi. Lavorare nel dominio delle frequenze permette di manipolare selettivamente queste componenti per ottenere risultati come denoising (rimozione del rumore), miglioramento dei bordi o compressione.

Dal punto di vista pratico:

- si usa `fft2` per calcolare la DFT 2-D e `fftshift` / o non usarlo a seconda della convenzione (attenzione: se non si shiftano i coefficienti le basse frequenze sono nei **quattro angoli** dell'array FFT, se si usa `fftshift` la frequenza nulla è al centro);
- per visualizzare lo spettro è utile mostrare sia il modulo $|F(u,v)|$ sia $\log(|F|+1)$ per far emergere coefficienti a bassa ampiezza;
- filtri "ideali" a taglio netto (passa basso/passa alto con soglia netta) sono didattici ma causano artefatti (es. ringing) — filtri a transizione morbida (es. gaussiani) sono spesso preferibili in pratica;
- un metodo alternativo è selezionare coefficienti in base all'**energia** (soglia su $|F|^2$) per ottenere una forma di compressione spettrale.



Riassunto Esercitazione 9

Obiettivo dell'esercitazione Mostrare come la trasformata di Fourier 2-D permette di analizzare e filtrare immagini separando il contenuto in bande di frequenza, e sperimentare tecniche di filtraggio (passa basso, passa alto, filtraggio basato sull'energia) per capire effetti pratici su qualità, dettaglio e compressione.

Take-home messages generali

- La DFT 2-D è uno strumento potente per isolare e manipolare componenti spettrali; la visualizzazione logaritmica è essenziale per vedere i dettagli deboli.
- Filtri passa basso rimuovono dettagli/rumore ma sfocano; filtri passa alto enfatizzano bordi ma aumentano rumore e artefatti.
- Tagli netti nello spettro causano ringing; preferire maschere morbide (es. gaussiane) per applicazioni pratiche.
- La soglia energetica (conservare coefficienti con $|F|^2 > \text{soglia}$) è una semplice forma di compressione spettrale: funziona bene se poche componenti contengono la maggior parte dell'energia, ma la scelta della soglia (mean vs percentile) è critica.
- Attenzione alla convenzione di shift (low freq al centro o ai bordi): mantenere coerenza tra visualizzazione e manipolazione dei coefficienti.



Image Filtering

1. Computer Vision

- È ovunque: social media, articoli, libri, ecc.
- **Obiettivo:** replicare il sistema visivo umano.
- Estrae informazioni da immagini/video digitali per definire attributi utili.

2. Immagini digitali

- Le immagini si possono vedere come funzioni che associano a ogni posizione (x, y) un valore di pixel.
- **Immagini in scala di grigi:** valori di intensità tra 0 e 255.
- **Immagini a colori (RGB):** tre canali (rosso, verde, blu) con valori interi 0-255 → più di 16 milioni di combinazioni possibili.

3. Image Filtering

- Operazione che modifica i valori dei pixel (range), non le posizioni.
- Obiettivi:
 - **Denoising** (riduzione rumore)
 - **Super-resolution**
 - **In-painting**
 - **Estrazione di informazione** (bordi, angoli, blobs).

4. 2D Discrete Fourier Transform (DFT)

- Analisi delle immagini nel dominio delle frequenze.
- Permette di separare componenti a **bassa e alta frequenza**, utile per il filtraggio.

5. Low-Pass Filter

- Lascia passare solo le basse frequenze, elimina le alte.
- Formula:

$$H(u, v) = \begin{cases} 1 & \text{se } D(u, v) \leq D_0 \\ 0 & \text{altrimenti} \end{cases}$$

con $D(u, v) = \sqrt{(u - M/2)^2 + (v - N/2)^2}$.

- Utile per ridurre rumore e rendere l'immagine più "smooth".

6. High-Pass Filter

- Lascia passare solo le alte frequenze, elimina le basse.
- Formula:

$$H(u, v) = \begin{cases} 0 & \text{se } D(u, v) \leq D_0 \\ 1 & \text{altrimenti} \end{cases}$$

- Utile per evidenziare bordi e dettagli fini.

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
from scipy import fftpack as f
import numpy.fft as f
```

✦ Esercizio 1. Digital images and 2-Dimensional DFT

(a) Leggere l'immagine `barbara.png` (scaricabile dalla cartella del corso) utilizzando il comando:

```
np.array(plt.imread('barbara.png'), dtype='float64'),
```

(b) Visualizzare l'immagine utilizzando `imshow` con l'opzione `cmap='gray'` per visualizzarla in scala di grigi.

(c) Scrivere una funzione chiamata `VisFourier` che:

- effettui la trasformata di Fourier bidimensionale dell'immagine con `fft2` e usi `fftshift` per centrare la frequenza nulla;
- visualizzi prima il valore assoluto dei coefficienti della trasformata e successivamente il `log(|F| + 1)`.

```
In [3]: # -----
# Funzione per visualizzare la FFT 2D
# -----
def VisFourier(I, title_prefix=""):
    """
    Visualizza modulo e log(|F|+1) della FFT 2D di un'immagine.
    """
    F = f.fftshift(f.fft2(I))
    mag_F = np.abs(F)

    fig, axs = plt.subplots(1, 2, figsize=(10, 4))
    axs[0].imshow(mag_F, cmap='gray')
    axs[0].set_title(f'{title_prefix} |F(u,v)|')
    axs[0].axis('on')

    axs[1].imshow(np.log1p(mag_F), cmap='gray')
    axs[1].set_title(f'{title_prefix} log(|F(u,v)|+1)')
    axs[1].axis('on')

    plt.show()

# -----
# Filtri
# -----
def lowpass_filter(I):
    """
    Filtro passa basso con soglia N/4
    """
    F = f.fft2(I)
    N = int(np.floor(I.shape[0] / 4))
    F_lp = np.zeros_like(F, dtype=complex)
    F_lp[:N, :N] = F[:N, :N]
    F_lp[-N:, -N:] = F[-N:, -N:]
    F_lp[-N:, :N] = F[-N:, :N]
    F_lp[:N, -N:] = F[:N, -N:]
    return f.ifft2(F_lp)

def lowpass_filter_alpha(I, alpha):
    """
    Filtro passa basso con parametro alpha in [0,1]
    """
    F = f.fft2(I)
    N = int(np.floor(I.shape[0] * alpha / 2))
    F_lp = np.zeros_like(F, dtype=complex)
    if N > 0:
        F_lp[:N, :N] = F[:N, :N]
        F_lp[-N:, -N:] = F[-N:, -N:]
        F_lp[-N:, :N] = F[-N:, :N]
        F_lp[:N, -N:] = F[:N, -N:]
    return f.ifft2(F_lp)

def highpass_filter_alpha(I, alpha):
    """
    Filtro passa alto con parametro alpha
    """
    F = f.fft2(I)
    N = int(np.floor(I.shape[0] * alpha / 2))
```

```

F_hp = np.copy(F)
if N > 0:
    F_hp[:N, :N] = 0
    F_hp[-N:, -N:] = 0
    F_hp[-N:, :N] = 0
    F_hp[:N, -N:] = 0
return f.ifft2(F_hp)

def newfilter(I):
    """
    Mantiene solo i coefficienti con energia > energia media
    """
    F = f.fft2(I)
    mean_en = np.mean(np.abs(F)**2)
    mask = np.abs(F)**2 > mean_en
    print(f"Compression ratio = {np.sum(mask) / I.size:.2%}")
    F_f = np.zeros_like(F, dtype=complex)
    F_f[mask] = F[mask]
    return f.ifft2(F_f)

```

```

In [ ]: # ESERCIZIO 1 - Digital images and 2D DFT
print("Esercizio 1 - Digital images and 2D DFT")

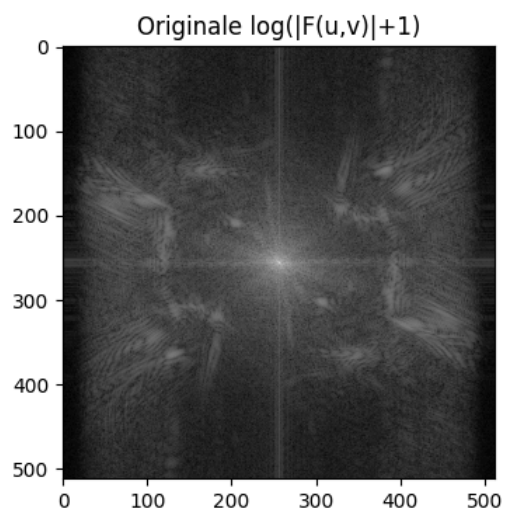
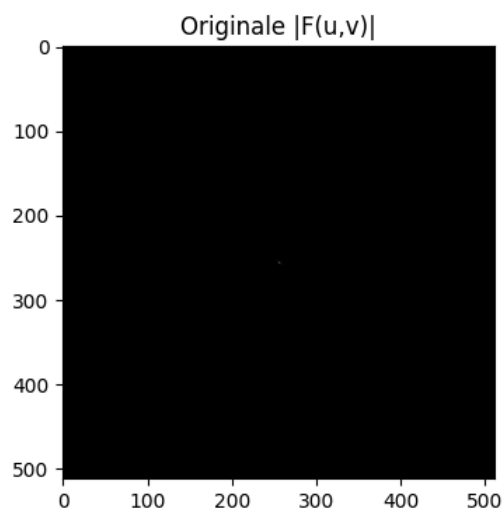
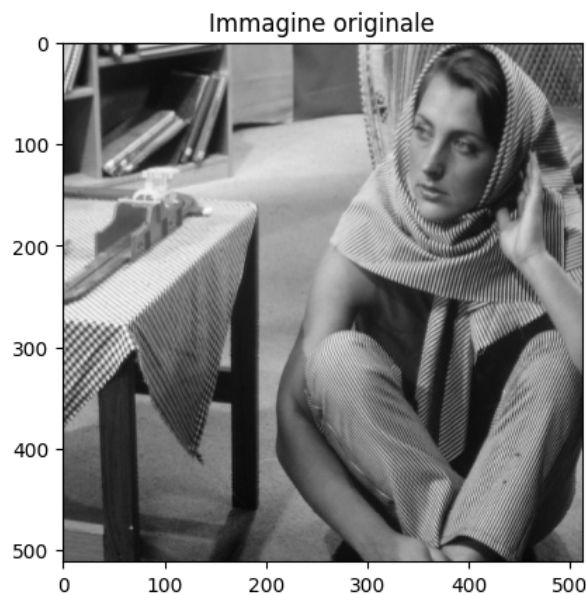
I = np.array(plt.imread('barbara.png'), dtype='float64')

# (a) Visualizzazione immagine
plt.imshow(I, cmap='gray')
plt.title('Immagine originale')
plt.axis('on')
plt.show()

# (b-c) FFT dell'immagine
VisFourier(I, "Originale")

```

Esercizio 1 - Digital images and 2D DFT



COMMENTO.

Obiettivi

- Leggere e visualizzare l'immagine `barbara.png`.
- Calcolare la FFT 2-D e visualizzare $|F(u,v)|$ e $\log(|F|+1)$ tramite `VisFourier`.

Commento (plot e risultati)

- `VisFourier` mostra il modulo e la versione logaritmica: ci aspettiamo una concentrazione di energia nelle frequenze basse (spot centrale se `fftshift` è usato, o nei quattro angoli se non si è shiftato). [La visualizzazione logaritmica rende visibili anche i contributi a bassa ampiezza.](#)

Take-home message

- Verificare sempre la convenzione di shift; usare `log(|F|+1)` per visualizzare dettagli dello spettro; la forma dello spettro dà subito informazioni su quanto l'immagine è dominata da basse frequenze (texture morbida) o da alte frequenze (molti dettagli).

✦ Esercizio 2. Low pass filtering

(a) Scrivere una funzione che prenda in input un'immagine e applichi un filtro passa basso. Data un'immagine di dimensione $NimesN$, il filtro passa basso azzeri tutti i coefficienti corrispondenti alle frequenze superiori a $N/4$ e inferiori a $-N/4$ nelle due dimensioni. La funzione restituisce in output l'immagine filtrata.

(b) Utilizzare la funzione creata al punto precedente per applicare il filtro passa basso all'immagine `barbara.png` e visualizzare il risultato ottenuto. Utilizzare `VisFourier` per visualizzare anche i coefficienti della trasformata di Fourier dell'immagine filtrata.

(c) Scrivere una funzione simile alla precedente in cui si inserisca un parametro $\alpha \in [0, 1]$ che controlli quante frequenze conservare: se $\alpha = 1$ si conservano tutti i coefficienti, se $\alpha = 0$ si azzerano tutti i coefficienti.

(d) Applicare la funzione creata al punto precedente all'immagine `barbara.png` per i valori $\alpha = [0, 0.05, 0.1, 0.2, 0.7]$ e visualizzare i risultati ottenuti (sia l'immagine filtrata che i coefficienti della trasformata).

```
In [ ]: # ESERCIZIO 2 - Low pass filtering
print("Esercizio 2 - Low pass filtering")

# (a-b) Passa basso N/4
I_lp = lowpass_filter(I)
VisFourier(I_lp, "LP N/4")
plt.imshow(np.real(I_lp), cmap='gray')
plt.title('Immagine filtrata LP N/4')
plt.axis('on')
plt.show()

# (c-d) Passa basso con alpha
alpha_vals = [0, 0.05, 0.1, 0.2, 0.7]
for alpha in alpha_vals:
    I_lpa = lowpass_filter_alpha(I, alpha)
    VisFourier(I_lpa, f"LP α={alpha}")
    plt.imshow(np.real(I_lpa), cmap='gray')
    plt.title(f'Immagine filtrata LP α={alpha}')
    plt.axis('on')
    plt.show()
```

Esercizio 2 - Low pass filtering

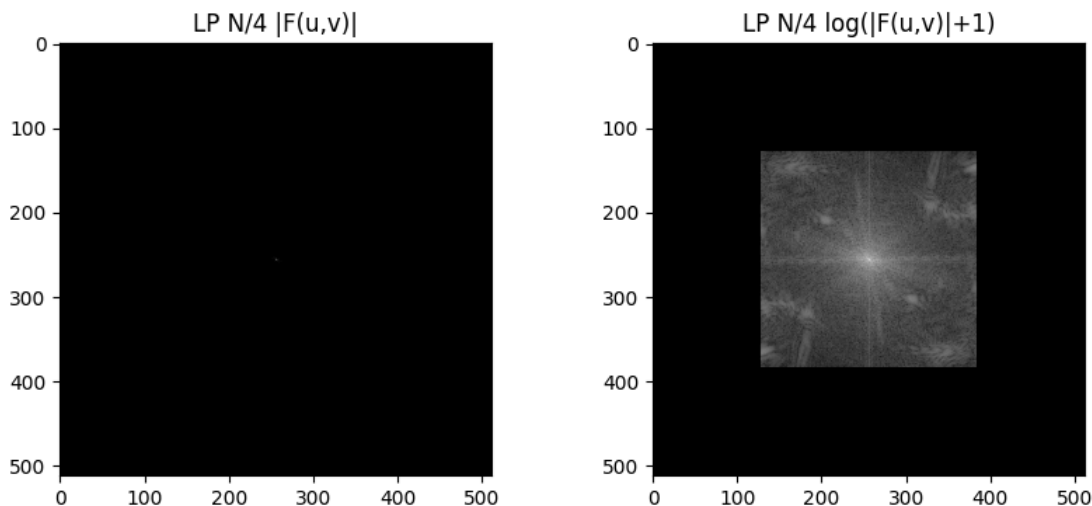
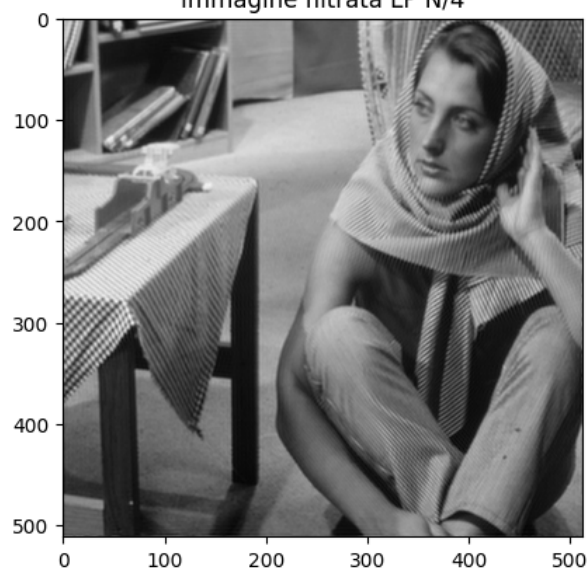
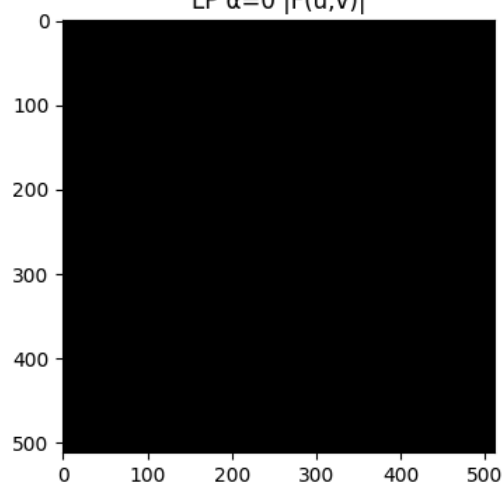


Immagine filtrata LP N/4



LP $\alpha=0$ $|F(u,v)|$



LP $\alpha=0$ $\log(|F(u,v)|+1)$

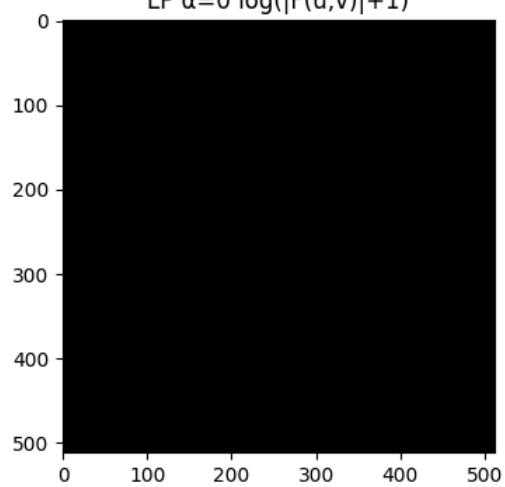
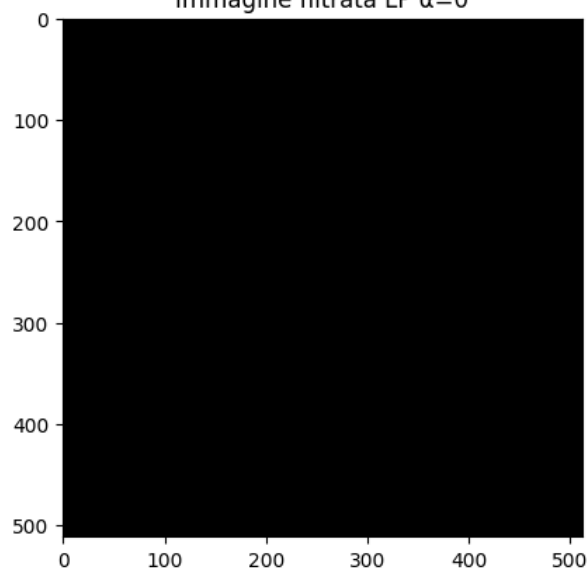
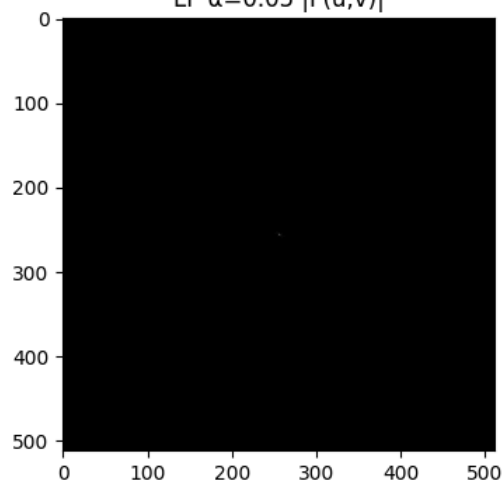


Immagine filtrata LP $\alpha=0$



LP $\alpha=0.05$ $|F(u,v)|$



LP $\alpha=0.05$ $\log(|F(u,v)|+1)$

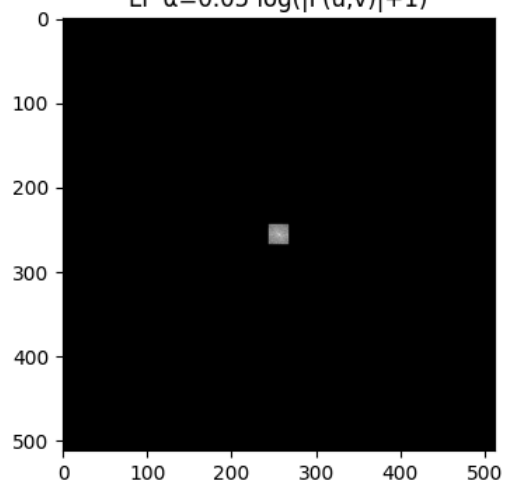
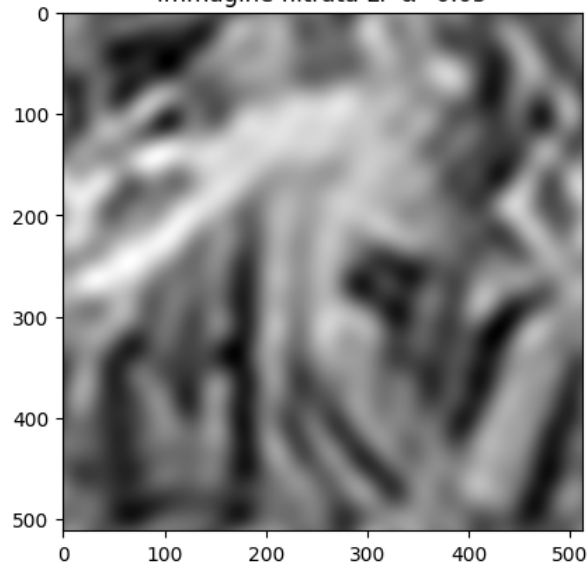
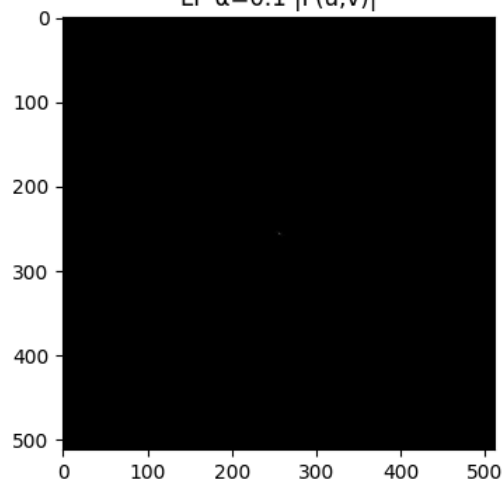


Immagine filtrata LP $\alpha=0.05$



LP $\alpha=0.1$ $|F(u,v)|$



LP $\alpha=0.1$ $\log(|F(u,v)|+1)$

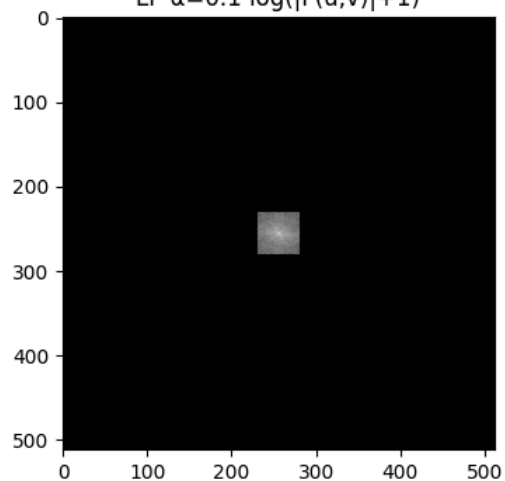
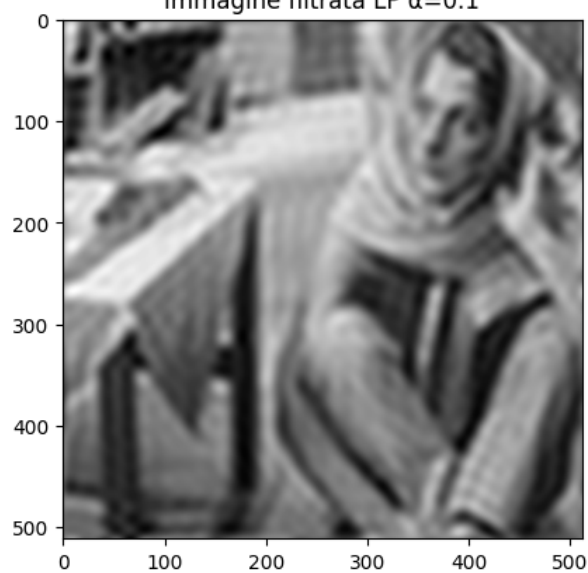
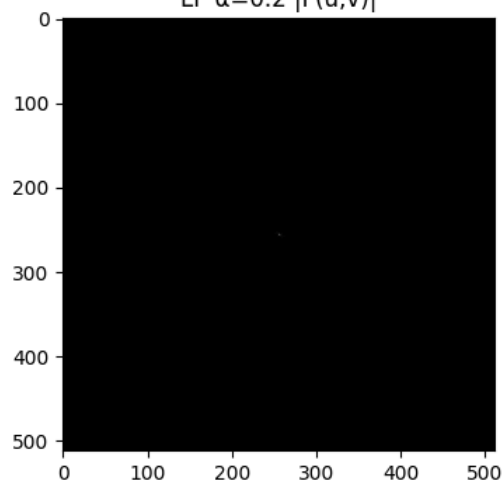


Immagine filtrata LP $\alpha=0.1$



LP $\alpha=0.2$ $|F(u,v)|$



LP $\alpha=0.2$ $\log(|F(u,v)|+1)$

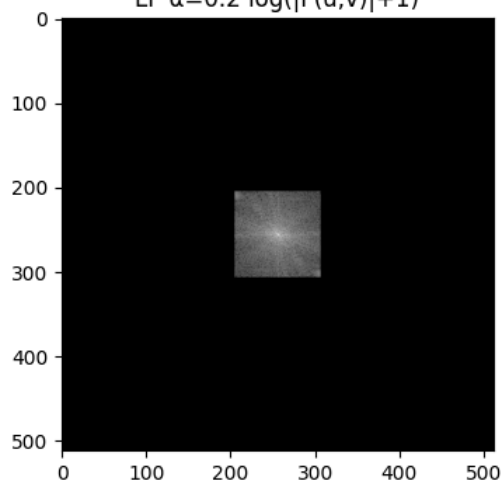
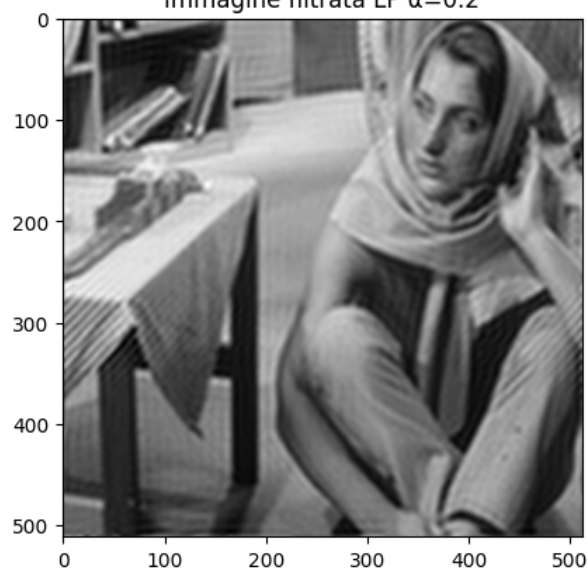
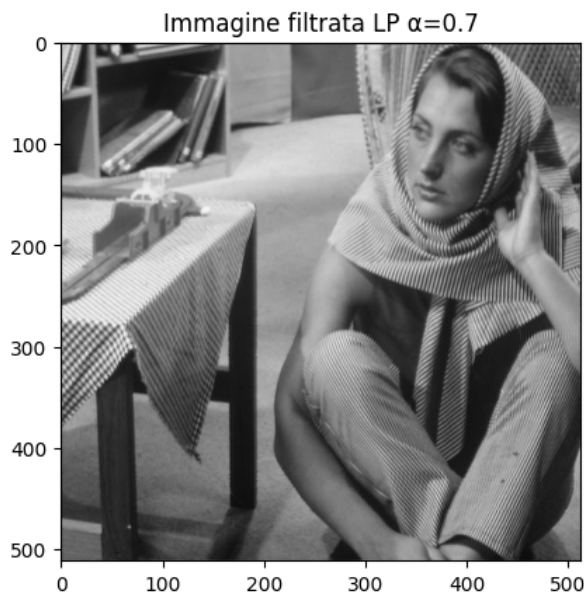
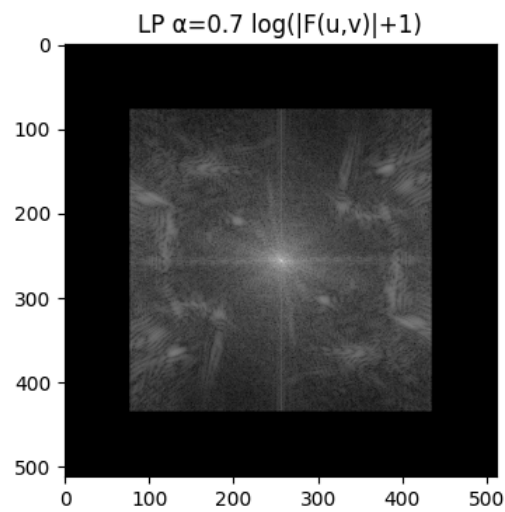
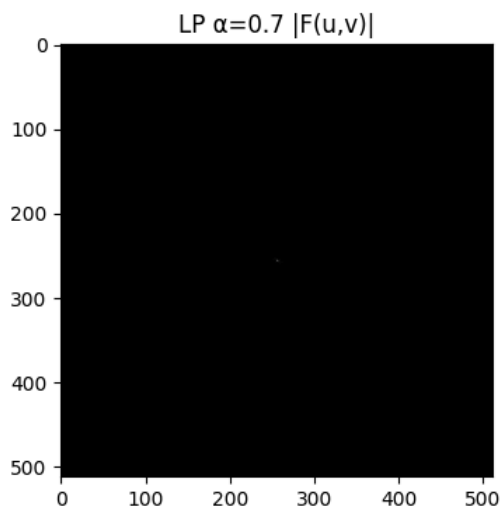


Immagine filtrata LP $\alpha=0.2$





COMMENTO.

Obiettivi

- Implementare un filtro passa basso (taglio delle alte frequenze) e una versione parametrizzata con `alpha` che controlli la quantità di frequenze conservate.
- Applicare il filtro a `barbara.png` per vari valori di `alpha` e visualizzare i risultati e gli spettri.

Commento (plot e risultati)

- Risultati attesi per i plot immagine: con `alpha` piccoli l'immagine diventa molto sfocata; con `alpha` $\rightarrow 1$ si ripristina l'immagine originale.
`alpha` = [0, 0.05, 0.1, 0.2, 0.7].
- Nota implementativa importante: la funzione `lowpass_filter` usa `fft2` **senza** `fftshift` e copia blocchi ai quattro angoli (questa è coerente con la FFT non shiftata, dove le basse frequenze stanno agli angoli). Se invece si usa `fftshift` bisogna creare una maschera centrata. È importante essere coerenti: visualizzare con `fftshift` ma modificare senza shift porta a confusione/maschere sbagliate.
- Nel dominio spettrale la maschera passa basso si traduce in una regione centrale (o angoli, a seconda della convenzione) non nulla; nello spettro log si noterà la rimozione delle componenti esterne. Aspettarsi inoltre artefatti da taglio netto (bande/contorni nelle immagini filtrate).

Take-home message

- `Alpha` controlla gradualmente il trade-off dettaglio $\leftrightarrow$ smoothing.
- Mantenere coerenza tra `fftshift` /non shift e la creazione della maschera.
- Per applicazioni pratiche usare transizioni morbide (es. finestra gaussiana) per ridurre artefatti.

✦ Esercizio 3. High pass filtering

(a) Scrivere una funzione che applichi un filtro passa alto. Data un'immagine di dimensione $NimesN$ e un parametro α , il filtro passa alto azzeri tutti i coefficienti corrispondenti alle frequenze comprese tra $-\alpha \cdot N/2$ e $\alpha \cdot N/2$ nelle due dimensioni.

(b) Applicare la funzione creata al punto precedente all'immagine `barbara.png` per i valori α = [0, 0.05, 0.1, 0.2, 0.7] e visualizzare i risultati ottenuti (sia l'immagine filtrata che i coefficienti della trasformata).

(c) Provare un metodo alternativo di filtraggio in dominio di Fourier che elimini i coefficienti meno espressi, ad esempio i coefficienti la cui energia è minore dell'energia media, e visualizzare il risultato.

```
In [ ]: # ESERCIZIO 3 - High pass filtering
print("Esercizio 3 - High pass filtering")
```



```
# (a-b) Passa alto con alpha
for alpha in alpha_vals:
    I_hpa = highpass_filter_alpha(I, alpha)
    VisFourier(I_hpa, f"HP  $\alpha$ ={alpha}")
    plt.imshow(np.real(I_hpa), cmap='gray')
    plt.title(f'Immagine filtrata HP  $\alpha$ ={alpha}')
    plt.axis('on')
    plt.show()

# (c) Nuovo metodo di filtraggio
I_f = newfilter(I)
VisFourier(I_f, "Filtro energia > media")
plt.imshow(np.real(I_f), cmap='gray')
plt.title('Immagine filtrata - Energia > media')
plt.axis('on')
plt.show()
```

Esercizio 3 - High pass filtering

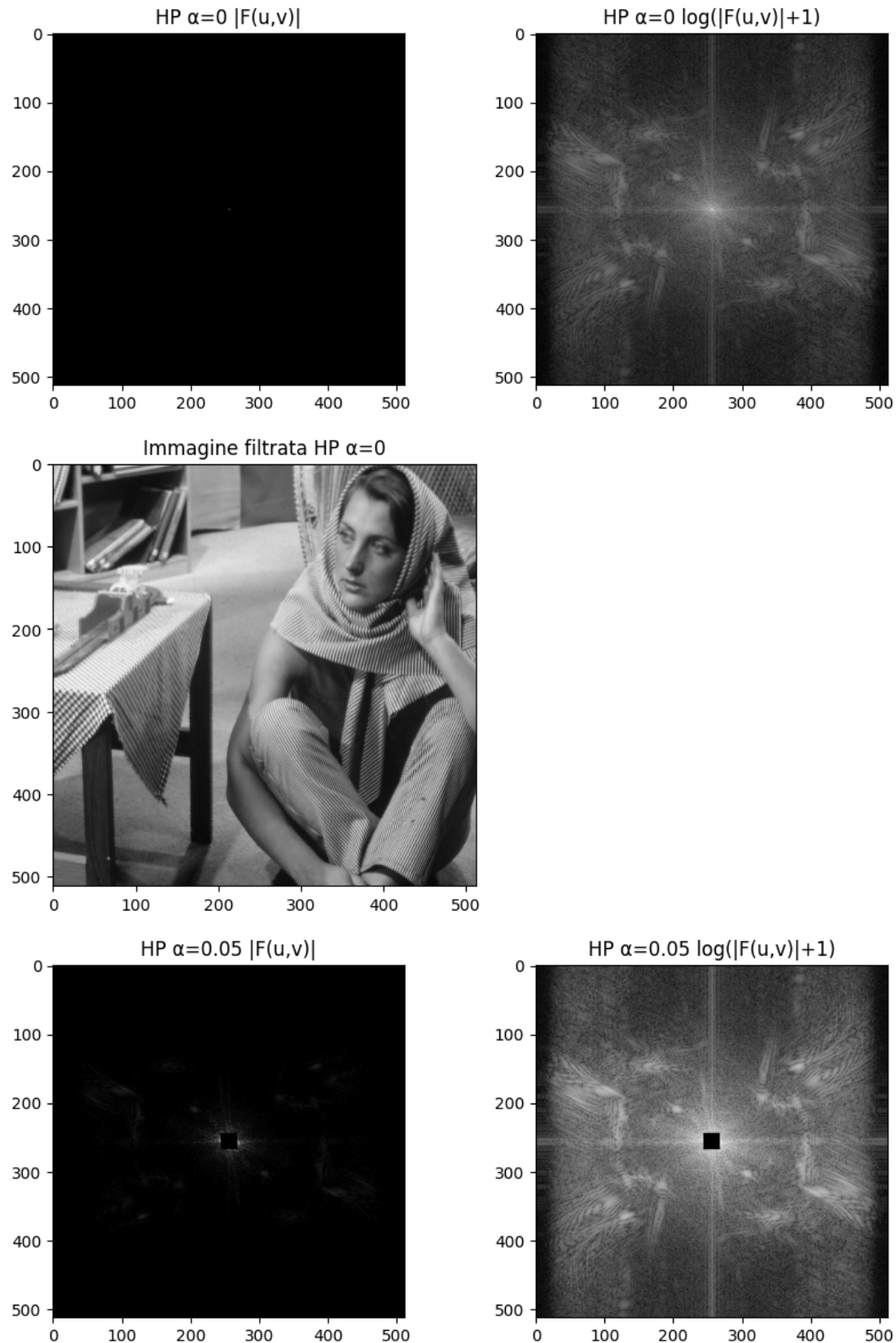
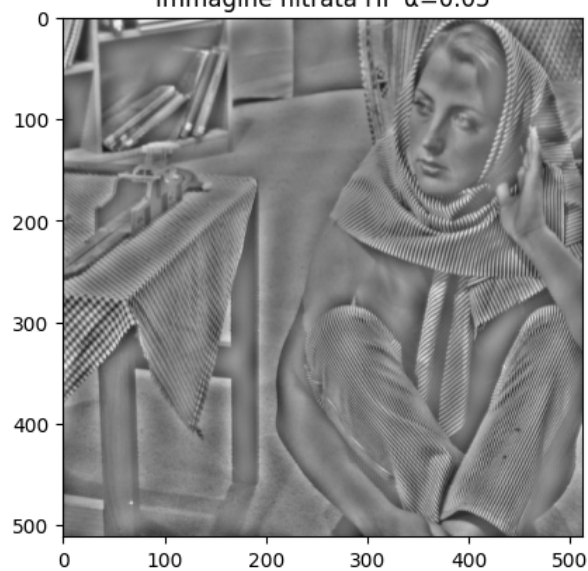
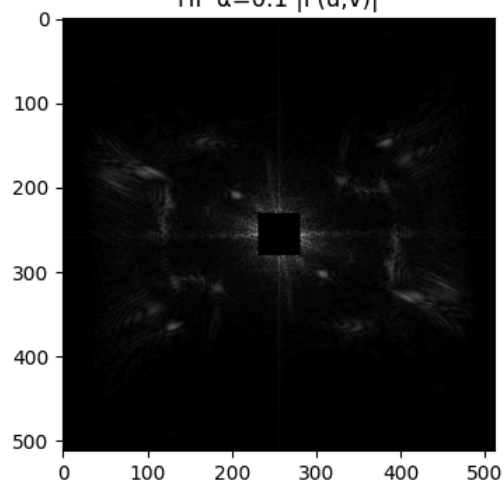


Immagine filtrata HP $\alpha=0.05$



HP $\alpha=0.1$ $|F(u,v)|$



HP $\alpha=0.1$ $\log(|F(u,v)|+1)$

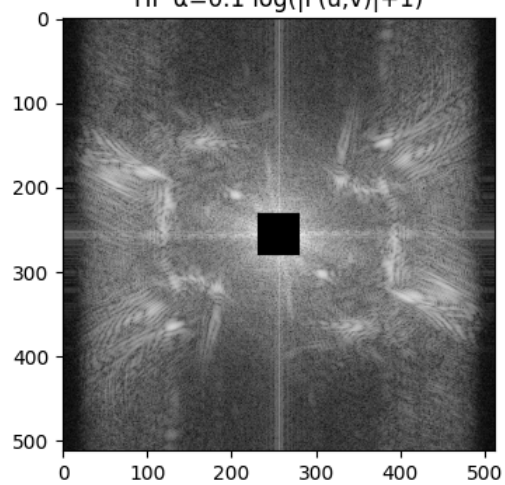
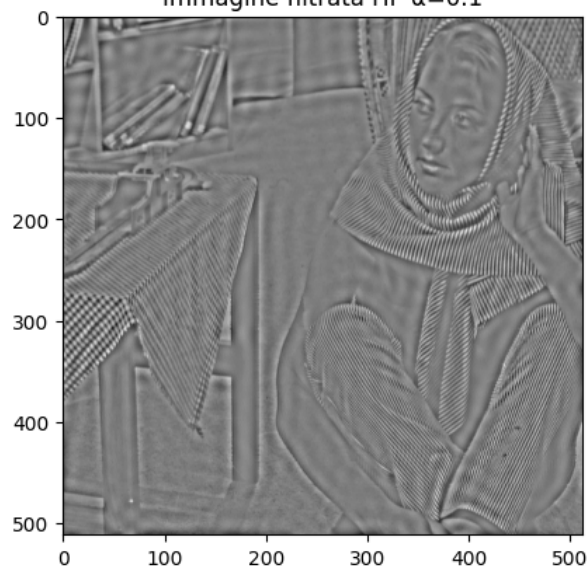
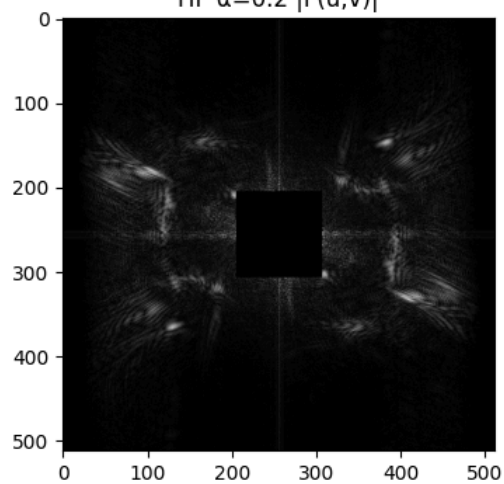


Immagine filtrata HP $\alpha=0.1$



HP $\alpha=0.2$ $|F(u,v)|$



HP $\alpha=0.2$ $\log(|F(u,v)|+1)$

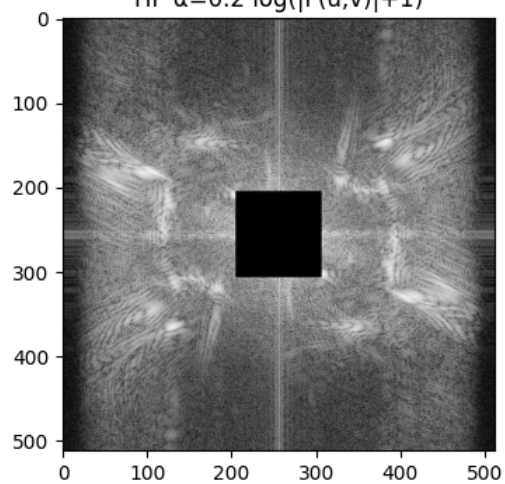
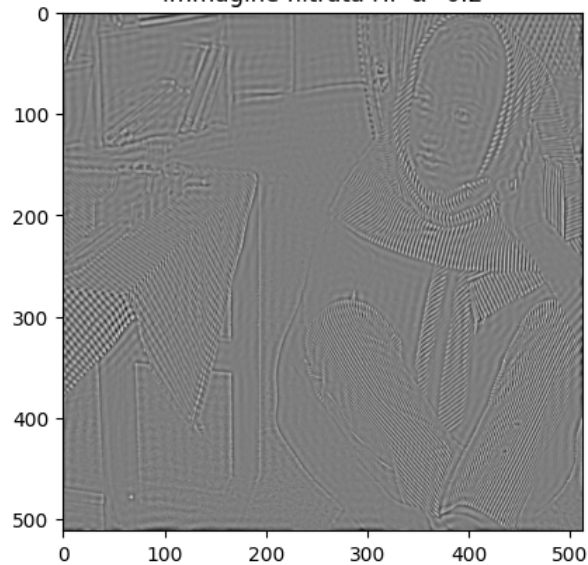
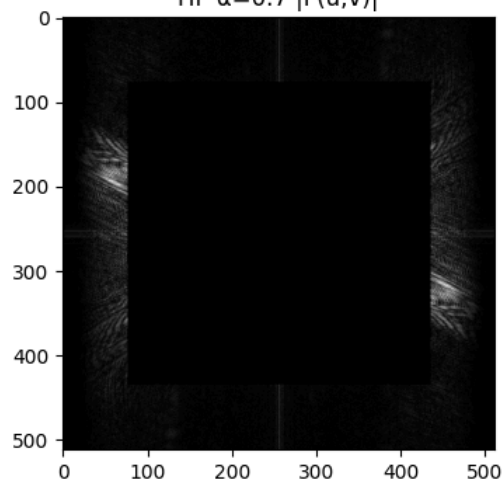


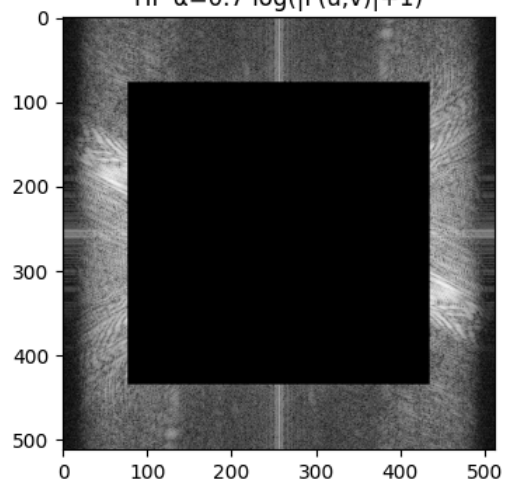
Immagine filtrata HP $\alpha=0.2$

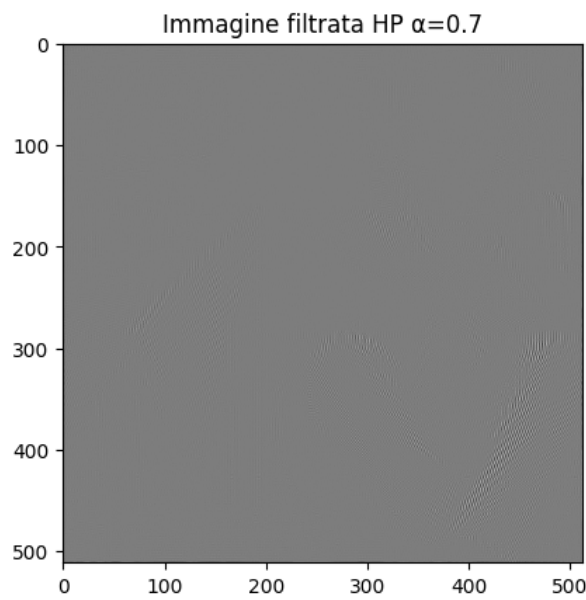


HP $\alpha=0.7$ $|F(u,v)|$

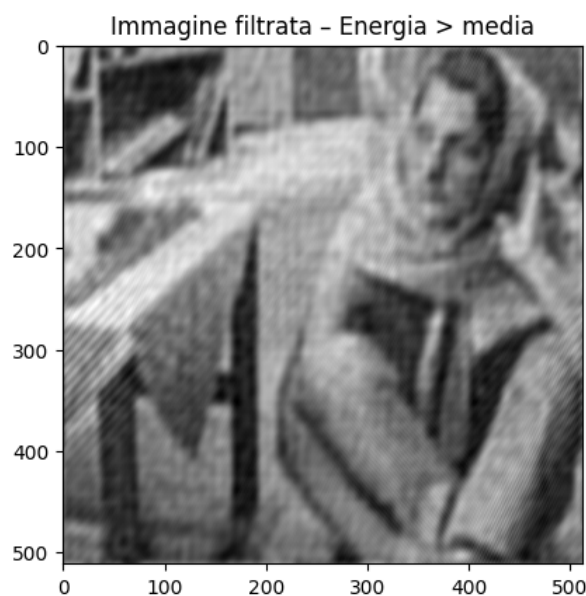
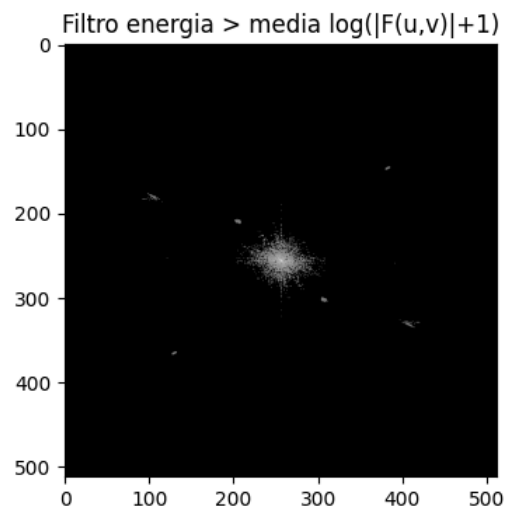
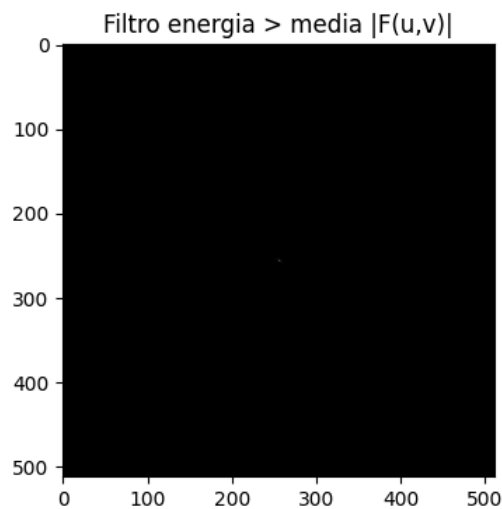


HP $\alpha=0.7$ $\log(|F(u,v)|+1)$





Compression ratio = 0.82%



COMMENTO.

Obiettivi

- Implementare un filtro passa alto parametrico (α) e confrontarne l'effetto per diversi α .
- Implementare un filtro che conserva solo i coefficienti con energia > energia media e valutare il risultato (compressione).

Commento (plot e risultati)

- Con `highpass_filter_alpha`, man mano che α aumenta la fascia di frequenze azzerata cresce intorno alle basse frequenze: l'immagine ricostruita enfatizzerà i bordi e i dettagli (aspetto "edge-like"), ma aumenterà il rumore e le strutture ad alta frequenza. I plot spettrali mostreranno la rimozione delle componenti centrali.
- Il metodo `newfilter` (energia > media) stampa `Compression ratio = 0.82%` — cioè viene mantenuta una porzione molto piccola dei coefficienti (0.82%), il che indica una ricostruzione molto sparsa. Ciò può portare a una immagine ricostruita che conserva gli elementi più energetici (spesso strutture dominanti) ma perde molte sfumature; inoltre dipende fortemente dalla soglia scelta (media vs percentile).

Take-home message

- I filtri passa alto sono utili per evidenziare dettagli/bordi ma aumentano il rumore; bilanciare con tecniche di smoothing se necessario.
- La soglia energetica è una tecnica semplice per compressione spettrale: funziona se pochi coefficienti portano la maggior parte dell'energia, ma usare la media come soglia può essere troppo aggressivo in molti casi — considerare percentile o soglie adattive.
- Valutare sempre la qualità della ricostruzione con metriche (RMSE, PSNR) oltre alla sola ispezione visiva.